

Tecnologie Web

Nicolò Giuliani

Indice

1	I set di caratteri	3
2	Markup	5
2.1	Formati per strutture di dati	5
2.2	Linguaggi di markup	6
3	HTML	8
3.1	Attributi globali	14
4	CSS	15
4.1	Cascading Style Sheet	16
4.2	Proprietà	18
5	Uniform Resource Identifier (URI)	18
6	HyperText Transfer Protocol	22
6.1	I metodi di HTTP	24
6.2	REST	26
7	Introduzione a Javascript	27
7.1	Tipi di dato	28
7.2	Oggetti Predefiniti	29
7.3	Json	29
7.4	Javascript client-side	30
7.5	Il Document Object Model (DOM)	31
7.6	Oggetti del DOM	31
7.7	DOM Node	32
7.8	DOM Element	32
7.9	Navigare sul DOM	33
8	Approfondimenti su HTTP	34
8.1	HTTP/2	35
8.2	HTTP/3	36
8.3	Cookies	36
8.4	Headers e struttura dei cookie	37
8.5	Cookie di sessione	38
8.6	Base 64	39
8.7	Header e Payload	39
8.8	Signature	40
8.9	CORS	40
8.10	HTTP Caching	40

1 I set di caratteri

Internet ha proposto il problema di rendere correttamente gli alfabeti di migliaia di lingue nel mondo. Bisogna utilizzare una rappresentazione digitale del testo, ovvero identificare gli elementi e lo spazio. Poi lo definiamo come standard.

- caratteri: accenti, maiuscolo/minuscolo, segni aggiuntivi, ecc...
- spazio:
 - ordine: alfabetico
 - contiguità: ogni valore numerico è associato ad un carattere
 - raggruppamento: a seconda del valore numerico si definiscono dei gruppi logici

A seconda del sistema numerale (bit / hexa) la matematica non cambia rimane invariata. bbbbbb Questo perché c'è una connessione diretta tra rappresentazione binaria e esadecimale dei numeri secondo cui ogni blocco di 4 cifre binarie (bit) corrisponde ad una cifra esadecimale.

$$1bit = 2^1 \text{ combinazioni : 2 valori}$$

$$2bit = 2^2 \text{ combinazioni : 4 valori}$$

$$4bit = 2^4 \text{ combinazioni : 16 valori}$$

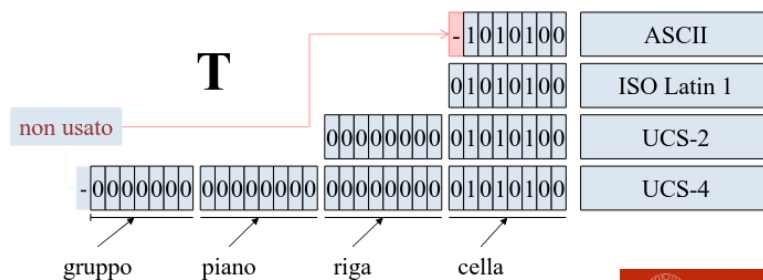
$$8bit = 2^8 \text{ combinazioni : 256 valori}$$

$$16bit = 2^{16} \text{ combinazioni : 65536 valori}$$

- **ASCII** definisce 128 caratteri (7bit), di cui 33 di controllo (NUL, ESC, TAB, ecc.), con ripetizioni, e 95 caratteri di alfabeto latino, maiuscole, minuscole, numeri, punteggiatura, ecc.
- EBCDIC (contemporaneo ad ASCII), creato dalla IBM per i mainframe, utilizza 8bit con 56 codici di controllo e le lettere dell'alfabeto NON sono continue.
- ISO 646-1991 sistema europeo dove si lasciavano 12 codici liberi per nazione per i propri fini, si sacrificavano questi caratteri "\$ @ ~ ' | " .
- Code page per ASCII: vari ampliamenti del vecchio standard per ampliare da 128 a 255, però non c'è modo se non esterno di capire quale code page è stata utilizzata.
- ISO 8859/1 (ISO Latin 1): una nuova sequenza basata su ASCII per i primi 128 caratteri ma che venivano ampliati a 256.
- **Unicode** e ISO/IEC 10646: due standard internazionali che si sono poi uniti (nel 2020) con gli stessi codici con codifiche a lunghezza variabile (UTF-8, UTF-16, ecc) e fissa (UCS-2, UCS-4). Nel 2025 si definiscono 160k caratteri.
 - Unicode include tutti i caratteri degli alfabeti in ordine originale, molto efficiente, caratteri semplici (no bold). Quando possibile due caratteri comuni vengono unificati, sistema perfettamente stabile (quando un carattere viene definito non vengono modificato) e per convertirli in altri formati è molto semplice.
 - UCS-2 è uno schema a piani di 2byte, mentre UCS-4 lo estende ed è uno schema di 31bit in 4byte.

I piani sono definiti così:

- Piano 0 (BMP o Basic Multilingual Plane): alfabeti moderni



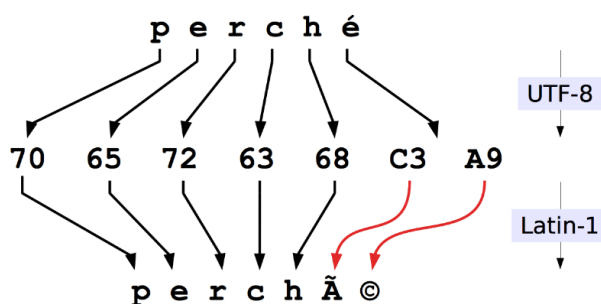
- Piano 1 (SMP o Supplementary Multilingual Plane): alfabeti antichi
- Piano 2 (SIP o Supplementary Ideographic Plane): ulteriori caratteri ideografici CJK (Chinese, Japanese, Korean) non presenti in BMP.
- Piano 3 (TIP o Tertiary Ideographic Plane): caratteri cinesi antichi
- Piano 4-13 sono riservati o inutilizzati
- Piano 14 (SSP o Supplementary Special-purpose Plane): Caratteri tag, in disuso
- Piani 15 e 16: Private Use Areas

Dato che utilizzare 4byte per carattere come in UCS è esagerato nella maggior parte dei casi si utilizza UTF, ovvero un sistema di lunghezza variabile.

- UTF-16: utilizzando 2048 valori a coppie surrogate con sequenze a 32bit ci permette di avere un code point dei piani 1-16.
- UTF-8: accede ad tutti i caratteri di UCS-4 ma con un numero variabile tra 1 e 4 byte.

Come posso capire se il sistema che utilizzo sia little-endian o big-endian (primo byte significativo)? Attraverso il Byte Order Mark (BOM) si aggiunge ovunque (non in mezzo) il carattere *FEFF* e si capisce che è big-endian, in caso opposto little-endian.

Tra UTF-8 e Latin 1 non ci sono differenze per i caratteri ASCII ma per le lettere speciali (accenti, ecc.) cambia, infatti UTF-8 utilizza 2byte mentre Latin 1 soltanto 1byte.



2 Markup

Strutture di dati

Come abbiamo studiato in precedenza esistono vari modi per organizzare i dati

- Record (tupla)
- Liste
- Tabelle
- Alberi
- Grafi

2.1 Formati per strutture di dati

Vediamo che formati possiamo trovare

- Tabelle
 - CSV: vecchio formato per i dati tabulari, la prima riga è il nome di un campo, una riga è definita dal ritorno a capo, il separatore di campo è dato da virgola, punto, barra verticale, ecc.
 - Excel / Libre office: XLSX è il formato di Microsoft
- Alberi
 - json: nativo per javascript, supporto su tutti i linguaggi, le parentesi graffe/quadre indicano i contenitori
 - yaml: basato come struttura di json ma influenzato da python
- Grafi
 - RDF: è un modello concettuale, utilizza URI, le informazioni possono essere raggruppate per soggetto
- Testo
 - plain text
 - Tex / LaTeX
 - Html: attuale base del web basato su tag, utilizza anche altri linguaggi come CSS e javascript
 - XML: come per html si utilizzano i tag, enfasi sul markup descrittivo e gerarchie
 - Markdown

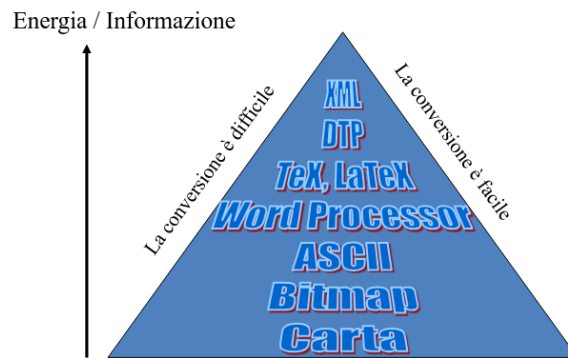
Cosa è il markup?

E' una qualsiasi forma di interpretazione del testo

Bisogna per prima cosa definire che esistono sia modi pubblici che privati, ovviamente il secondo caso è quando una azienda ne detiene i diritti per scopi commerciali quindi può modificarlo, aggiornarlo per qualsiasi motivo.

Esistono anche la distinzione fra binario e visibile, nel secondo caso un essere umano può modificarlo manualmente attraverso un formato leggibile, mentre nel primo caso il testo non è visibile.

Definiamo anche interno o esterno quando le istruzioni di presentazione sono in mezzo le parole o se separati in due blocchi di informazioni.



- Markup puntuazionale: consiste nell'usare un insieme prefissato di segni per fornire informazioni perlopiù sintattiche sul testo
- Markup presentazionale: il markup presentazionale consiste nell'indicare effetti (tipografici o altro) per rendere più chiara la presentazione del contenuto. Attraverso cambi di paragrafo o di pagina, interlinea, pallini per liste, ecc
- Markup procedurale: consiste nell'indicare con precisione ad un sistema automatico che effetto attivare e che procedura (serie di istruzioni) eseguire nella visualizzazione del contenuto.
- Markup descrittivo: consiste nell'identificare strutturalmente il tipo di ogni elemento del contenuto. Per esempio definendo se si tratta di un titolo, paragrafo, citazione, ecc.
- Markup referenziale: consiste nel fare riferimento ad entità esterne al documento per fornire significato o effetto grafico ad elementi del documento. (Es. "CdL"!diventa "Corso di Laurea")
- Metamarkup: regole di interpretazione del markup e permette di estendere o controllare il significato del markup
- Markup metabolizzato: maiuscole/minuscole, punteggiatura, spazi e ritorni a capo sono essi stessi elementi di markup

2.2 Linguaggi di markup

- TROFF/NROFF: utilizzato oggi giorno per la documentazione tecnica, comandi esterni/interni, definisce 4 tipi di carattere come "roman, italic, bold, symbol".
- Tex / LaTeX: generazione di macro che semplificano notevolmente la generazione di testi particolarmente sofisticati. Attraverso le librerie si semplifica molto gli argomenti complessi.
- Linguaggi wiki: usato per documentazione e knowledge base; adatto a non programmatori. Utilizzato da Wikipedia.
- Markdown: orientato alla leggibilità in testo puro, set ristretto di costrutti: titoli, enfasi, link, elenchi, code span, blocchi di codice, molto estendibile ed diffuso. Utilizzato ovunque (README, note, blog, ecc)
- json: ottimo per API e storage, non per il testo
- yaml: ottimo per file di configurazione e manifest

- SGML: un meta-linguaggio (standard) non proprietario di markup descrittivo

Un meta-linguaggio è un linguaggio per definire linguaggi, una grammatica di costruzione di linguaggi. Quindi SGML non è un linguaggio di markup, ma un linguaggio con cui definiamo linguaggi di markup.

Un documento in un linguaggio di markup definito sulla base di SGML è sempre composto delle seguenti tre parti

- Dichiarazione SGML (non obbligatorio)
 - Essa permette di specificare valori fondamentali come la lunghezza dei nomi degli elementi, il set di caratteri usati, le specifiche caratteristiche di minimizzazione ammesse, ecc.)
- Dichiarazione di documento (DOCTYPE)
 - Vengono cioè elencati [i file che contengono] gli elementi ammissibili, il contesto in cui possono apparire, ed altri eventuali vincoli strutturali
- Istanza del documento
 - L'istanza del documento è quella parte del documento che contiene il testo vero e proprio, dotato del markup appropriato. Esso contiene una collezione di elementi (tag), attributi, entità, PCDATA, commenti, ecc.

Un document di markup derivato da SGML (HTML, XML, ecc.) ha i seguenti elementi

- Elementi
 - Un elemento è individuato da un tag iniziale, un contenuto ed un tag finale. Non confondere elemento con TAG!
- Attributi
 - Gli attributi sono informazioni aggiuntive sull'elemento che non fanno effettivamente parte del contenuto
- Entità
 - Le entità sono frammenti di documento memorizzati separatamente e richiamabili all'interno del documento. Esse permettono di riutilizzare lo stesso frammento in molte posizioni
- Testo
- Commenti
 - Sono molto comodi per passare informazioni tra un autore e l'altro, o per trattenere informazioni per se stessi, nel caso le dimenticassimo.
- Processing instructions
 - sono elementi particolari (spesso di senso esplicitamente procedurale) posti dall'autore o dall'applicazione per dare ulteriori indicazioni su come gestire il documento XML nel caso specifico

XML 1.0 (1998) è definito come un sottoinsieme di SGML. XML permette di avere documenti senza l'obbligo di tutti i tag, elementi vuoti, entità definite, ecc. rispetto ad SGML.

3 HTML

Html nasce come linguaggio di markup per documenti ipertestuali e multimediali. Ora vediamo la struttura di un documento html:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Document title</title>
  </head>
  <body>
    <p>Text of a paragraph</p>
  </body>
</html>
```

analizziamo gli elementi fondamentali

- <!DOCTYPE html>: segnala il tipo di markup usato nel documento e la sua versione, con molte complessità e variazioni
- html: La radice dell'albero
- head: Informazioni globali sul documento. TAG:
 - <title>: il titolo del documento
 - <link>: link di documenti a tutto il documento
 - <script>: librerie di script
 - <style>: librerie di stili
 - <meta>: meta-informazioni sul documento
 - <base>: l'URL da usare come base per gli URL relativi
- body: Il contenuto del documento

Elementi inline

Si dividono in elementi fontstyle e elementi phrase. Molti sono deprecati, e si suggerisce comunque sempre di usare gli stili CSS.

- TT (TeleType, font monospaziato, ad es. Courier)
- I (corsivo)
- B grassetto
- U (sottolineato - deprecato)
- S e STRIKE (testo barrato - deprecato)
- BIG, SMALL (testo più grande e più piccolo)

Elementi di blocco

I tag di blocco definiscono l'esistenza di blocchi di testo che contengono elementi inline

- `<p>`: paragrafo
- `<div>`: Blocco generico
- `<pre>`: blocco preformattato
- `<address>`: l'autore della pagina
- `<blockquote>`: Citazione lunga

Blocchi con ruolo strutturale

- `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`: header

Elementi di lista

Le liste sono contenitori di elementi omogenei

- `<ul>`: Unordered list: lista non ordinata di elementi `<li>`
- Attributi: `type` (`disc`, `square`, `circle`)
- `<ol>`: ordered list: lista ordinata di elementi `<li>`
- Attributi: `start` (valore iniziale) e `type` (`1`, `a`, `A`, `i`, `I`)
- `<dl>`: lista di definizioni, composta di molti elementi `<dt>` (definition term) e `<dd>` (definition data)

Elementi generici

Sono elementi privi di caratteristiche semantiche o presentazionali predefinite, assumono quelle desiderate con l'aiuto dei loro attributi (`style`, `class`, `lang`).

- `<div>` mantiene la natura di elemento blocco, il resto è neutro
- `<span>` mantiene la natura di elemento inline, il resto è neutro

Effetti grafici minori

- `<hr>` (horizontal rule): una piccola riga orizzontale attraverso lo schermo. Si può controllarne la larghezza e lo spessore
- `<br>` (break): una andata a capo forzata (non spezza logicamente il paragrafo, impone semplicemente il cambio riga)

Elementi di struttura

HTML 5 introduce nuovi elementi con una semantica precisa, per organizzare il documento in contenitori che possono essere più facilmente manipolati, esportati, modificati, etc.

- `<main>`: la parte principale della pagina, ovvero un contenitore di testo o di sottosezioni che rappresenta la parte più importante della pagina. Al suo interno possiamo trovare:
 - `<section>`: un contenitore generico annidabile
 - `<article>`: una parte del documento self-contained e pensata per essere ri-usata e distribuita come unità atomica (post di un blog, news, feed, etc.)
- `<aside>`: una sezione collegata al testo ma separata dal flusso principale (note a margine, sidebar, incisi, pubblicità, etc.)
- `<header>`: contiene l'intestazione della sezione corrente o dell'intera pagina ed è solitamente usato per tabelle di contenuti, indici, form di ricerca, intestazioni
- `<footer>`: contiene la "parte conclusiva" della sezione corrente o dell'intera pagina. E' usato principalmente per mostrare informazioni (testuali, non metadati) sugli autori della pagina, copyright, produzione, licenze
- `<nav>`: liste di navigazione, sono molto utili per l'accessibilità, in quanto possono essere più facilmente identificate e accedute anche da utenti disabili (tramite screenreaders o altri ausili)

Link

I link sono definiti con elementi `<a>` (àncore nel documento, elemento inline), attributi:

- href: specifica l'URL della destinazione
- name: specifica un nome che può essere usato come ancora di destinazione di un link (non si vede che il testo è blu con un link, rimane nero)

Immagini

Le immagini inline sono definite con l'elemento `<img>`. Formati tipici: jpeg, gif, png.

- `<img>` è un elemento vuoto, completamente definito dai suoi attributi.
 - src (required): l'URL della risorsa contenente l'immagine
 - alt: testo alternativo se l'immagine non è mostrata
 - width: forza una larghezza in pixel per l'immagine.
 - height: forza una altezza in pixel per l'immagine

Attraverso l'attributo `srcset` è possibile indicare più risorse per la stessa immagine, da usare alternativamente. A seconda dello user agent scegliamo immagini con dimensioni (e quindi risoluzioni) diverse, e non dobbiamo affidarci al ridimensionamento algoritmico dell'immagine con l'inevitabile perdita di dettaglio

```

```

- L'attributo `srcset` indica quali risorse usare e la dimensione "reale" di ciascuna risorsa
- L'attributo `sizes` indica quali tipi di schermo associare a ciascuna risorsa
- L'attributo `src` è un fallback nel caso che il browser non conosca `srcset`

L'elemento `<figure>`

Le immagini sono dunque elementi vuoti inline. In molte situazioni ho bisogno di un elemento complesso, di tipo blocco, magari con didascalia e marginatura controllabile.

```
<h1>Figure</h1>
<figure>
  
  <figcaption>A cake</figcaption>
</figure>
```

Embedding

Permettono di arricchire la pagina HTML con molteplici tipi di contenuti alternativi.

- `<video>` per fare embedding di un video
- `<audio>` per fare embedding di un audio
- `<object>` per fare embedding di oggetti multimediali, vediamo quali:
 - `<canvas>`: possibilità di disegnare direttamente sulla pagina interagire con gli oggetti multimediali
 - `<video>`: meccanismo generico per il caricamento di file e stream video. Possibilità di avere diversi file, tra i quali il browser sceglie quello da eseguire.
 - `<audio>`: (uguale a video)
 - `<math>`
- `<embed>` elemento generico per fare embedding di oggetti multimediali. Un po' in disuso
- `<iframe>` per fare embedding di una pagina HTML all'interno di un'altra pagina HTML.

```
<iframe title="Mappa OpenStreetMap - Bologna Centro"
  width="300" height="200" loading="lazy"
  referrerpolicy="no-referrer"
  sandbox="allow-scripts allow-same-origin"
  src="https://www.openstreetmap.org/export/embed.html?bbox=11.320%2C44.490%2C11.370%2C44.500">
</iframe>
```

Tabelle

Le tabelle vengono specificate riga per riga. Di ogni riga si possono precisare gli elementi, che sono o intestazioni o celle normali. Una tabella può anche avere una didascalia, un'intestazione ed una sezione conclusiva. E' possibile descrivere insieme le caratteristiche visive delle colonne.

```
<table border="1" >
  <tr>
    <th>Salesman</th><th>Jan</th>
    <th>Feb</th><th>Mar</th>
  </tr>
  <tr>
    <th>John Smith</th> <td>12000</td>
    <td>13000</td><td>15000</td>
  </tr>
</table>
```

Possiamo anche definire *rowspan* e *colspan* per allargare le celle

```
<table border="1" >
  <tr>
    <td>First row <br> First collumn</td>
    <td colspan="2">First row <br> Second column</td>
  </tr>
  <tr>
    <td colspan="2">Second row <br> First column</td>
    <td rowspan="2">Secondrow <br> Thrird column</td>
  </tr>
  <tr>
    <td>Third row <br> First column</td>
    <td>Third row <br> Second column</td>
  </tr>
</table>
```

First row First column	First row Second column	
Second row First column		Second row Third column
Third row First column	Third row Second column	

Form

Con i FORM si utilizzano le pagine HTML per inserire valori che vengono poi elaborati sul server. I FORM sono legati ad applicazioni server-side. Crea una connessione HTTP con il server, specificando una ACTION (cioè un applicazione che funga da destinatario) a cui fare arrivare i dati. Il destinatario riceve i dati, li elabora e genera un documento di risposta, che viene spedito, tramite il server HTTP, al browser.

- <form>: il contenitore dei widget del form. Attributi:
 - method: il metodo HTTP da usare (GET, POST)
 - action: l'URI dell'applicazione server-side da chiamare

- `<input>`, `<select>`, `<textarea>`: i widget del form. Attributi:
 - name: il nome del widget usato dall'applicazione server-side per determinare l'identità del dato
 - type: il tipo di widget (input, checkbox, radio, submit, reset, etc.)
- `<button>`: un bottone cliccabile (diverso dal submit)
- `<label>`: la parte visibile del widget

HTML LS introduce molte novità per velocizzare, semplificare e controllare l'inserimento dei dati da parte dell'utente. L'oggetto *input* è arricchito con molti attributi che permettono di controllare il focus e aggiungere suggerimenti agli utenti:

- placeholder: contiene una stringa che sarà visualizzata in un campo di testo se il focus non è su quel campo
- required: il form non può essere sottomesso senza una valore per questo elemento
- readonly: è un elemento del form non modificabile
- list: il valore deve appartenere ad uno dei valori ammessi (specificati in un elemento `<datalist>` associato)

```
<input type="text" list="province">
<datalist id="province">
  <option value="BO">Bologna</option>
  <option value="MO">Modena</option>
  <option value="FC">Forli Cesena</option>
</datalist>
```

In HTML LS sono introdotti molti nuovi tipi per l'oggetto input. I browser forniscono widget diversi nell'interfaccia in base al tipo del campo:

- email: in fase di validazione il browser verifica se il testo inserito contiene il simbolo '@' e il dominio è (sintatticamente) corretto
- url: si verifica che il testo inserito segue le specifiche degli URL
- number: il browser visualizza bottoni per incrementare/decrementare il valore. E' possibile specificare il valore minimo, massimo e l'unità di modifica in altri attributi di input
- range: il browser visualizza uno slider per incrementare o decrementare un valore numerico. E' possibile specificare il valore minimo, massimo e l'unità di modifica
- date: richiede al browser la visualizzazione di un calendario tra cui selezionare una data. Esistono vari tipi collegati come month, week, time (orario), etc.
- search: testo renderizzato in modo diverso su alcuni browser (iPhone)
- color: usato per mostrare una tavolozza di colori, da cui selezionare un codice RGB

```

<p>Nome: <input name="nome" type="text" placeholder="Inserisci il tuo nome" size="40" autofocus></p>
<p>Cognome: <input name="cognome" size="40" placeholder="Inserisci il tuo cognome"></p>
<p>Email: <input name="mail" type="email"></p>
<p>Lezioni: <input type="number" min="5" max="25" step="1" value="1"></p>
<p>Livello: <input type="range" name="level" min="1" max="10" step="1" value="3"></p>
<p>Data Inizio: <input name="birth" type="date" value="2011-04-01"></p>
<p><input type="submit" value="Iscriviti"></p>

```

3.1 Attributi globali

Sono attributi globali quelli definiti su tutti gli elementi del linguaggio HTML. Essi costituiscono attributi di qualificazione e associazione globale degli elementi. Per lo più per CSS e link ipertestuali.

- Id: un identificativo unico (su tutto il documento)
- Class: una lista (separata da spazi) di nomi di classe (per attribuzione semantica e di stile CSS)
- Style: un breve stile CSS associato al singolo elemento
- Title: un testo secondario associato all'elemento (per accessibilità e informazioni aggiuntive)

```

<!DOCTYPE html>
<html>
  <head>
    <title>Table elements </title>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <style>
      #p1 { font-family: "Times New Roman", "Times", serif; }
      #p2, #p3 { font-family: "Verdana", "Arial", sans-serif; }
      .first { color: blue; }
      .second { color: green; }
      .special { font-weight: bold; }
      .notspecial { font-weight: normal; }
    </style>
  </head>
  <body>
    <p id="p1" class="first special">This is the first paragraph of a short text.</p>
    <p id="p2" class="second">This is the <span id="s1" class="first">second</span>
    paragraph of the same text.</p>
    <p id="p3" class="special">This is the <span id="s2" class="notspecial"
    title="terzo!">third</span> paragraph of the same text.</p>
  </body>
</html>

```

Colori

In molte situazioni (sfondi, caratteri, ecc.) è possibile specificare un colore. HTML fornisce due modi per farlo:

- Codice RGB

- Nome

Tuttavia HTML 4 depreca l'uso esplicito di colori nel documento HTML, e suggerisce di usare i fogli di stile.

```
<font color="#FF0000">testo in rosso</font>
<body bgcolor="#008080">sfondo teal</body>
<td bgcolor="yellow">sfondo giallo</td>
```

Lunghezze

HTML usa le lunghezze per tutti gli oggetti con una presenza sulla pagina di dimensioni determinate (immagini, tabelle, frame, ecc.) Si usano tre tipi di lunghezze:

- Pixel: una dimensione in punti di schermo. È un numero assoluto

```

```

- Percentuali: una dimensione in proporzione alla dimensione del contenitore. È un numero seguito da un carattere

```
<table width="100%">...</table>
```

- Multi-lunghezze: una sequenza di valori di lunghezza. In questo caso il carattere '*' divide la quantità restante in parti uguali (dopo aver tolto le lunghezze esplicitate in pixel e percentuale).

```
<colgroup width="150,25%,*,*">
```

4 CSS

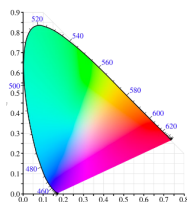
La **tipografia** è la disposizione armoniosa di tipi al fine di creare testo leggibile e piacevole alla vista sulla pagina e sugli schermi digitali.

Font e type face

Un font è una collezione di forme di caratteri integrate armoniosamente per le necessità di un documento in stampa. Tipicamente contiene forme per lettere alfabetiche, numeri, punteggiatura e altre caratteri standard nello scritto. Un type face (o font-family) è uno stile unico di caratteri che viene espanso in molti font di dimensione e peso e stili diversi.

Spazi colori

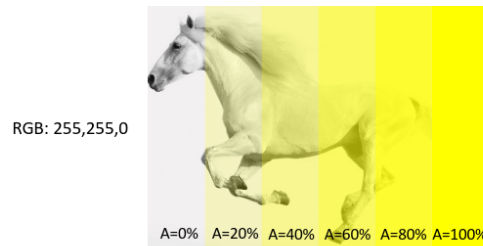
L'occhio umano ha tre tipi di coni, cellule incaricate di riconoscere colori. Assorbono frequenze diverse della luce riflessa sugli oggetti, che vengono interpretati separatamente dando l'impressione di colori diversi.



RGB è uno spazio colore additivo basato sull'identificazione di Rosso, Verde e Blu come colori primari. Poiché i coni dell'occhio sono sensibili separatamente alle frequenze del rosso, del verde e del blu, questo è un modello ragionevolmente vicino a quello dell'occhio umano.

	0, 0, 0		255, 0, 255		255, 127, 0
	255, 255, 255		0, 255, 255		127, 64, 0
	255, 0, 0		0, 255, 0		127, 127, 0
	255, 255, 0		0, 0, 255		127, 127, 127

RGBA è uno spazio colore derivato da RGB, a cui viene aggiunta una quarta dimensione, chiamata canale alpha.



CMYK

CMYK è uno spazio colore sottrattivo basato sull'identificazione di quattro colori primari, Cyan, Giallo, Magenta e Key (che è un nero molto scuro).



Si usa quattro colori invece di tre perché l'inchiostro nero fornisce una definizione dei colori molto migliore, aumentando il contrasto e riducendo la quantità di inchiostro colorato necessario per le tinte più scure.

4.1 Cascading Style Sheet

Come si utilizza? In HTML abbiamo

- posizionato presso il tag di riferimento

```
<html>
  <head>
    <title>Esempio CSS</title>
  </head>
  <body style="background-color:yellow;">
    <h1 class="title" style="color:blue;">
      I CSS: questi sconosciuti
    </h1>
    <p>Un paragrafo senza stile</p>
    <p id="p1" style="color:red;">
      Ecco un primo esempio di uso dei CSS.
```

```

    </p>
  </body>
</html>

```

- posizionato nel tag <style>

```

<html>
  <head>
    <title>Esempio CSS</title>
    <style type="text/css">
      body { background-color: yellow; }
      p { color: black; }
      .title { color: blue; }
      #p1 { color: red; }
    </style>
  </head>
  <body>
    <h1 class="title">
      I CSS: questi sconosciuti
    </h1>
    <p>Un paragrafo senza stile</p>
    <p id="p1">
      Ecco un primo esempio di uso dei CSS.
    </p>
  </body>
</html>

```

- indicato dal tag <link>

```

<html>
<head>
  <title>Esempio CSS</title>
  <link type="text/css"
    rel="stylesheet"
    href ="/style/external.css" />
</head>
<body>
  <h1 class="title">
    I CSS: questi sconosciuti
  </h1>
  <p>Un paragrafo senza stile</p>
  <p id="p1">
    Ecco un primo esempio di uso dei CSS.
  </p>
</body>
</html>

```

external.css



```

body {
  background-color: yellow;
}

p {
  color: black;
}

.title {
  color: blue;
}

#p1 {
  color: red;
}

```



Inoltre HTML permette di assegnare gli stili agli elementi in tre modi

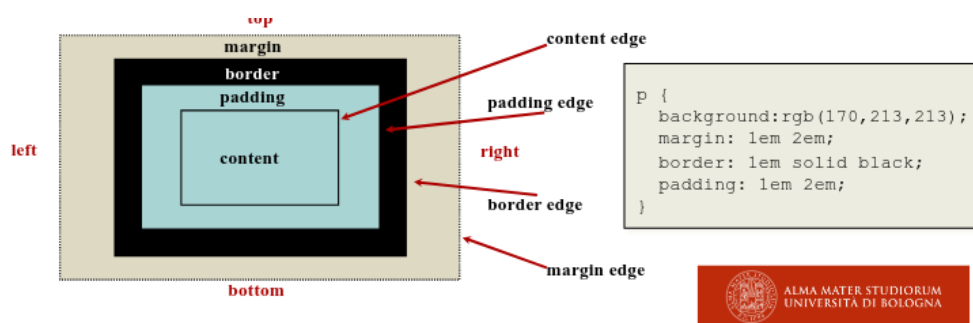
- assegnati a tutti gli elementi di un certo tipo (il nome dell'elemento)
- assegnati a tutti gli elementi di una certa categoria (il valore dell'attributo class)
- assegnati ad uno specifico elemento (identificato dal valore dell'attributo id)

4.2 Proprietà

La scatola

La visualizzazione di un documento con CSS avviene identificando lo spazio di visualizzazione di ciascun elemento.

- Margin: la regione che separa una scatola dall'altra, sempre trasparente (ovvero ha il colore di sfondo della scatola contenitore)
- Border: la regione ove visualizzare un bordo per la scatola
- Padding: la regione di respiro tra il bordo della scatola ed il contenuto. Ha sempre il colore di sfondo del contenuto
- Content: la regione dove sta il contenuto dell'elemento

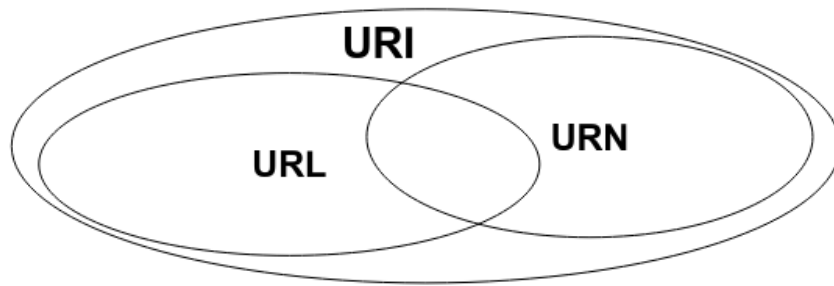


5 Uniform Resource Identifier (URI)

Attraverso gli URI, il WWW è stato in grado di identificare risorse accessibili tramite il proprio protocollo, HTTP, e tramite tutti gli altri protocolli esistenti. Gli URI (Uniform Resource Identifier) sono una sintassi usata in WWW per definire i nomi e gli indirizzi di oggetti (risorse) su Internet.

Gli Uniform Resource Identifier (URI) sono, per definizione:

- Locator (URL): una sintassi che contiene informazioni immediatamente utilizzabili per accedere alla risorsa (ad esempio, il suo indirizzo di rete)
 - Gli URL sono un indirizzo della risorsa che possa essere immediatamente utilizzato da un programma per accedervi.
- Names (una volta chiamati URN): una sintassi che permetta una etichettatura permanente e non ripudiabile della risorsa, indipendentemente dal riportare informazioni sull'accesso.
 - I names sono un nome stabile e definitivo di una risorsa, che possa fornire un'informazione certa ed affidabile sulla sua esistenza ed accessibilità.
 - I names debbono essere trasformati da un apposito servizio, negli URL attualmente associati alla risorsa. Inoltre la mappa deve essere aggiornata ogni volta che la risorsa viene spostata.



Nella visione moderna, la distinzione tra locator e name è secondaria rispetto al concetto di schemi: ogni URI appartiene ad uno schema (la parte della stringa che precede i due punti).

Criteri di design degli URI

Gli URI sono progettati per essere:

- Trascrivibili: gli URI sono sequenze di caratteri tratti da un set molto limitato (lettere, numeri, alcuni - pochi - caratteri speciali), per permettere la trascrizione degli URI su canali anche non elettronici (un pezzo di carta, un cartellone, ecc.)
- Fornire identificazione, non interazione: le operazioni eseguibili sulle risorse ed i protocolli per eseguirli non sono indicati nell'URI, ma nell'interpretazione che degli URI si fa all'interno dello specifico schema.
- Fornire spazi di nomi organizzati gerarchicamente: i caratteri “:”, “/”, “?” e “#” separano ambiti diversi all'interno degli URI. Il primo, onnipresente ed obbligatorio, è lo schema di interpretazione del resto. Schemi gerarchici usano il carattere “/” per separare livelli di specificità diversi del nome locale. “?” separa la parte di identificazione della risorsa da qualunque parametro necessario per accedervi, e “#” separa l'identificativo della risorsa da quello del frammento interno a cui si fa riferimento.

$$\text{URI} = \text{schema} : [// \text{ authority}] \text{ path } [? \text{ query}] [\# \text{ fragment}]$$

Analizziamo nel dettaglio

- Schema: protocollo TCP utilizzato
- La presenza esplicita di un'authority individua un'organizzazione gerarchica dello spazio dei nomi a cui sono delegati i nomi (che possono essere ulteriormente delegati)
 - authority = [userinfo @] host [: port]
- La parte path è la parte identificativa della risorsa all'interno dello spazio di nomi identificato dallo schema e (se esistente) dalla authority.
- La parte query individua un'ulteriore specificazione della risorsa all'interno dello spazio di nomi identificato dallo schema e dall'URI precedente. Di solito questi sono parametri passati all'URI (un processo) per specificare un risultato dinamico
- La parte fragment individua una risorsa secondaria (una risorsa associata, dipendente o in molti casi un frammento) della risorsa primaria.

Route

Una route è un'associazione della parte path di un URI ad una risorsa gestita o restituita da un server web.

- Managed route: il server associa ogni URI ad una risorsa o attraverso il file system locale (risorse statiche) oppure generate attraverso una computazione (risorse dinamiche)

```
var router = require("express").Router();

function getName(req, res) {
    res.send("<p>Fabio</p>");
}

function getEmail(req, res) {
    res.send("fabio.vitali@unibo.it");
}

router.get("/name", getName);
router.get("/email", getEmail);
app.use("/css", express.static('css'))
```

URI della richiesta	Risorsa restituita
http://www.example.com/name	Fabio
http://www.example.com/email	fabio.vitali@unibo.it
http://www.example.com/css/style.css	contenuto del file <code>css/style.css</code>

- File-system route: il server associa la radice della parte path ad una directory del file system locale e ogni filename valido all'interno di quella directory genera un URI corretto e funzionante.

Organizzazione del file system	URI disponibili
/	-
var/	-
www/	-
index.html	http://www.example.com/
alice/	-
index.html	http://www.example.com/alice/
bruce/	-
index.html	http://www.example.com/bruce/
...	...
fabio/	-
index.html	http://www.example.com/fabio/
pages/	
doc1.html	http://www.example.com/fabio/pages/doc1.html
doc2.html	http://www.example.com/fabio/pages/doc2.html
index.html	http://www.example.com/fabio/pages/
img/	
img1.gif	http://www.example.com/fabio/img/img1.gif
img2.jpg	http://www.example.com/fabio/img/img1.gif
css/	
style.css	http://www.example.com/fabio/css/style.css

Uri references (uriref)

Un URI assoluto contiene tutte le parti predefinite dal suo schema, esplicitamente precisate. Un URI reference fa sempre riferimento ad un URI di base (ad esempio, l'URI assoluto del documento ospitante l'URI reference) rispetto al quale fornisce porzioni differenti.

HTTP e HTTPS

I protocolli più usati nel WWW. HTTPS prevede una crittografia in entrambi i sensi del contenuto del messaggio.

FILE

Dà accesso ai file di un file system locale (cioè del computer su cui gira il browser)

`file://host/path [#fragment]`

MS Windows accetta anche "\", che però non è nello standard approvato!

DATA

Uno schema non gerarchico, che non fa riferimento ad una risorsa, ma CONTIENE la risorsa: tutti i dati della risorsa sono inseriti nell'URI vero e proprio.

`data:[<media type>|;base64|,<data>`

Si può codificare il dato, di solito usando base64;

FTP

`ftp://[user[:password]@]host[:port]/path`

Content Delivery Network (CDN)

In breve, una rete fortemente distribuita di server commerciali che collaborano tra loro per distribuire in maniera omogenea contenuti di grande successo senza inutili duplicazioni di occupazione e trasmissione di file.

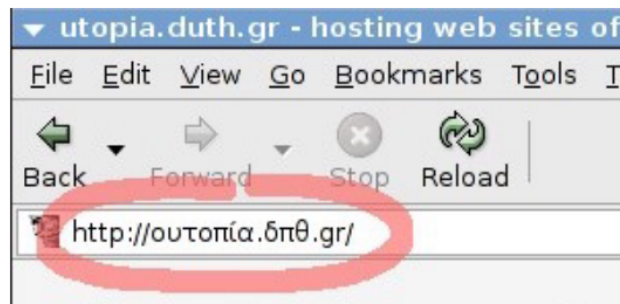
Ad esempio ogni utente che naviga su molti siti web, che usano le stesse librerie importanti ma non le prendono da un CDN, deve scaricare tutte le volte gli stessi identici file. Se questi siti invece fanno riferimento a librerie sullo stesso CDN, esse verranno scaricate una volta sola e rimarranno in cache anche per gli altri siti.

IRI

Internationalized Resource Identifier, RFC 3987 di Gennaio 2005, fornisce una sintassi per inserire caratteri di UCS-4 in un URI e per risolverli. In realtà non tutti i caratteri sono ammessi: tutti i codici di controllo e tutti i codici non corrispondenti a caratteri usati nelle lingue non sono ammessi.

IDN

Internazionalized Domain Name (IDN), o anche Internazionalized Domain Name for Application (IDNA), è uno standard IETF per estendere i nomi di dominio a tutti i caratteri non ASCII di Unicode (RFC 5890). Questi nuovi nomi di dominio vengono detti IDN.



CURIE

I Compact URI sono un modo per esprimere in maniera compatta famiglie di URI che condividono lo stesso prefisso

[prefix:curie]

Il prefisso è associato (in uno di molti modi, ad esempio XML namespaces che vedremo) ad un URI assoluto che deve essere sostituito nel CURIE per ottenere l'URI cercato

```
<html xmlns:dom="http://www.dominio.com/">
  <p>Si veda <a href="[dom:doc1]">doc 1</a></p>
</html>
```

In questo caso, il CURIE [dom:doc1] viene risolto nell'URI <http://www.dominio.com/doc1>

Linked Data

Il nome Linked Data è stato proposto da Tim Berners-Lee nel 2009 come modello concettuale per la caratterizzazione di tutte le risorse di dati che possono essere messi a disposizione ed incrociati da applicazioni ed umani. Per essere Linked Data, una collezione di dati deve:

- Usare URI per identificare oggetti.
- Usare HTTP URI in modo che questi oggetti possano essere referenziati e cercati da persone e user agent.
- Fornire informazioni utili sull'oggetto quando la sua URI è dereferenziata, usando formati standard come RDF.
- Includere link ad altre URI relative ai dati esposti per migliorare la ricerca di altre informazioni relative nel Web.

6 HyperText Transfer Protocol

HTTP è un protocollo client-server, generico e stateless utilizzato non solo per lo scambio di documenti Web ma in molte applicazioni distribuite.

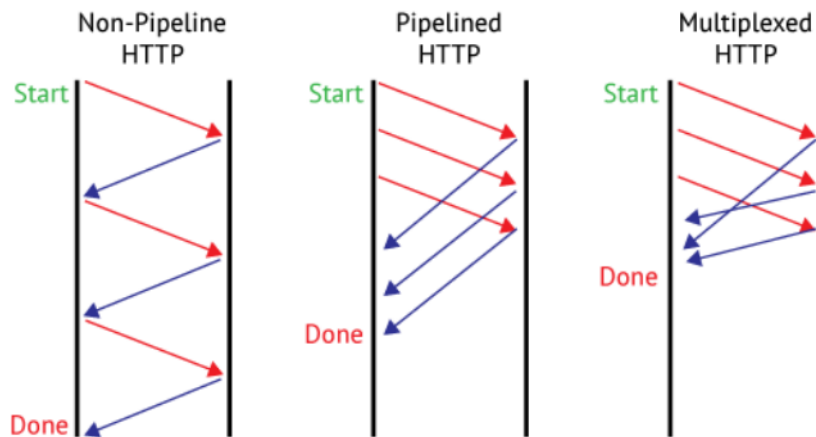
Una connessione HTTP è composta da una serie di richieste ed una serie corrispondente di risposte:

- Pipelining: trasmissione di più richieste senza attendere l'arrivo della risposta alle richieste precedenti. MA le risposte sono restituite nello stesso ordine delle richieste

- Multiplexing: nella stessa connessione è possibile avere richieste e risposte multiple, restituite anche in ordine diverso rispetto alle richieste e "ricostruite" nel client

HTTP/2 ha introdotto molte novità per migliorare le performance, tra cui:

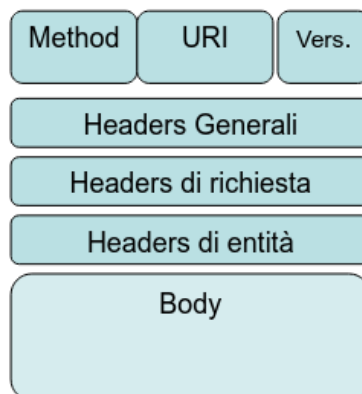
- PUSH: il server anticipa le richieste successive del client
- Compressione degli header: i dati non sono trasmessi in chiaro ma compressi (e in parallelo)



La richiesta HTTP

La richiesta HTTP si compone di:

- Method: azione del server richiesta dal client
- URI: identificativo della risorsa locale al server
- Version: numero di versione di HTTP
- Header: sono linee RFC822 divise in header generali, header di entità e header di richiesta
- Body: un messaggio MIME



Un esempio di richiesta

GET /beta.html HTTP/1.1

Referer: http://www.alpha.com/alpha.html

Connection: Keep-Alive

User-Agent: Mozilla/4.61 (Macintosh; I; PPC)

Host: www.alpha.com:80

Accept: image/gif, image/jpeg, image/png, */*

Accept-Encoding: gzip

Accept-Language: en

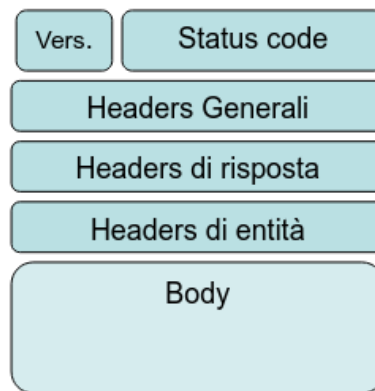
Accept-Charset: iso-8859-1,*,utf-8

6.1 I metodi di HTTP

- GET: metodo più frequente (sicuro ed idempotente), ed è quello che viene attivato facendo click su un link ipertestuale di un documento HTML, o specificando un URL nell'apposito campo di un browser per richiedere una risorsa ad un server
- HEAD: simile (sicuro ed idempotente) al metodo GET, ma il server deve rispondere soltanto con gli header relativi, senza il corpo. Viene usato per verificare validità, accessibilità e coerenza in cache di un URI
- POST: serve per trasmettere informazioni dal client al server relative alla risorsa identificata nell'URI (spedire i dati di un form HTML)
- PUT: trasmettere delle informazioni (idempotente) dal client al server, creando o sostituendo la risorsa specificata. L'argomento del metodo PUT è la risorsa che ci si aspetta di ottenere facendo un GET in seguito con lo stesso nome
- DELETE: rimuovere le informazioni (idempotente e NON sicuro) connesse con una risorsa
- PATCH: aggiornare parzialmente (NON idempotente NON sicuro) la risorsa identificata dall'URI. Usato per indicare quindi modifiche incrementali e non sovrascrivere risorse al server, come ad esempio nel caso di PUT
- OPTIONS: verificare opzioni, requisiti e servizi di un server, senza implicare che seguirà poi una altra richiesta

La risposta HTTP si compone di

- Status code: indica se la richiesta è andata a buon fine o meno
- Version: numero di versione di HTTP
- Header: (uguale ad richiesta)
- Body: un messaggio MIME



Un esempio di risposta
GET /index.html HTTP/1.1
Host: www.cs.unibo.it:80

```
HTTP/1.1 200 OK
Date: Fri, 26 Nov 2007 11:46:53 GMT
Server: Apache/1.3.3 (Unix)
Last-Modified: Mon, 12 Jul 2007 12:55:37 GMT
Accept-Ranges: bytes
Content-Length: 3357
Content-Type: text/html
<HTML> .... </HTML>
```

Vediamo ora due proprietà importanti

- **Sicurezza:** un metodo è sicuro se non genera cambiamenti allo stato interno del server, può essere eseguito da un nodo intermedio
- **Idempotenza:** l'effetto sul server di più richieste identiche è lo stesso di quello di una sola richiesta, può essere ri-eseguito da più agenti o in più tempi diversi senza effetti negativi

Status code

Lo status code è un numero di tre cifre, di cui la prima indica la classe della risposta, e le altre due la risposta specifica

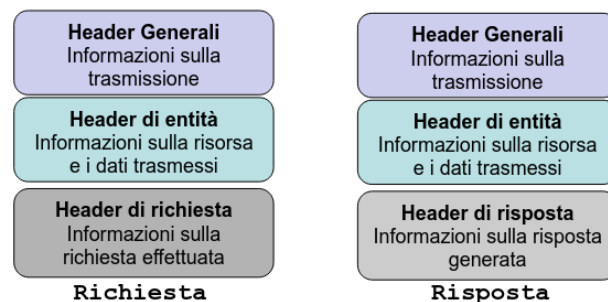
- **1xx:** Informational (Una risposta temporanea alla richiesta, durante il suo svolgimento)
- **2xx:** Successful (server ha ricevuto, capito e accettato la richiesta)
- **3xx:** Redirection (La richiesta è corretta, ma sono necessarie altre azioni da parte del client per portare a termine la richiesta)
- **4xx:** Client error (la richiesta del client non può essere soddisfatta per un errore da parte del client)
- **5xx:** Server error (la richiesta può anche essere corretta, ma il server non riesce a soddisfarla per un problema interno)

Esempi di status code

100 Continue (se il client non ha ancora mandato il body)
200 Ok (GET con successo)
201 Created (PUT con successo)
301 Moved permanently (URL non valida, il server conosce la nuova posizione)
400 Bad request (errore sintattico nella richiesta)
401 Unauthorized (manca l'autorizzazione)
403 Forbidden (richiesta non autorizzabile)
404 Not found (URL errato)
500 Internal server error (tipicamente un errore nel codice server-side)

header

Gli header sono righe di testo (RFC822) che specificano informazioni aggiuntive



Header generali: si applicano solo al messaggio trasmesso e si applicano sia ad una richiesta che ad una risposta, ma non necessariamente alla risorsa trasmessa.

Header dell'entità: danno informazioni sul body del messaggio, o, se non vi è body, sulla risorsa specificata.

Header della richiesta: sono posti dal client per specificare informazioni sulla richiesta e su se stesso al server.

Header della risposta: sono posti dal client per specificare informazioni sulla risposta e su se stesso al client.

API REST

Un API Web descrive un'interfaccia HTTP che permette ad applicazioni remote di utilizzare i servizi di dell'applicazione

6.2 REST

Acronimo di REpresentational State Transfer, ed è il modello architetturale che sta dietro al World Wide Web e in generale dietro alle applicazioni web "ben fatte" secondo i progettisti di HTTP.

Un'applicazione REST si basa fundamentalmente sull'uso dei protocolli di trasporto (HTTP) e di naming (URI) per generare interfacce generiche di interazione con l'applicazione, e fortemente connesse con l'ambiente d'uso.

L'obiettivo è creare API consistenti, predicibili e facili da capire e usare

L'architettura si basa su:

- Definire **risorsa** ogni concetto rilevante dell'applicazione Web

- Associargli un **URI** come **l'identificatore** e selettore primario
- Usare i verbi HTTP per esprimere ogni **operazioni** dell'applicazione secondo il modello CRUD
 - Creazione di un nuovo oggetto (metodo PUT)
 - Visualizzazione dello stato della risorsa (metodo GET)
 - Cambio di stato della risorsa (metodo POST)
 - Cancellazione di una risorsa (metodo DELETE)
- Esprimere in maniera parametrica ogni **rappresentazione dello stato interno della risorsa**, personalizzabile dal richiedente attraverso un **Content Type** preciso

Descrivere una RESTful API: Una API è RESTful se utilizza i principi REST nel fornire accesso ai servizi che offre

- end-point (URI / route) che supporta separando collezioni e elementi singoli
- metodi HTTP di accesso GET, PUT, POST, DELETE
- rappresentazioni in *Input* e *Output*
- condizioni di errore e i messaggi che restituisce in questi casi

Il modello CRUD

Pattern tipico delle applicazioni di trattamento dei dati

- Create: inserimento nel database di un nuovo record
- Read: accesso in lettura al database
- Update
- Delete

7 Introduzione a Javascript

Nato nel 1995 con il nome di JavaScript (prima ECMAScript) introdotto da Sun e Netscape. Standardizzato da ECMA International nel 1997. Per 18 anni sono state fatte piccole estensioni e modifiche. Nel giugno 2015 è stata approvata la versione 6 sono state introdotte molte differenze, molti nuovi costrutti. **Come eseguire uno script?**

- Client-side: eventi
 - Ogni elemento del documento ha alcuni eventi associati (click, mouseover, doppio click, tasto di tastiera, ecc), e degli attributi on+evento associati.
 - inserendo istruzioni JavaScript (o chiamata a funzione) nel valore dell'attributo si crea una chiamata callback.
- Server-side: routing
 - I servizi server-side sono associati a URI. Il modo più antico e semplice è creare servizi diversi e inserirli in file separati, ciascuno con un URI proprio. Aprendo una connessione all'URI, viene invocato lo script ed eseguito il servizio.

Come eseguire uno script su browser?

- In maniera sincrona, appena lo script viene letto, in un tag `<script>` o in un file
- In maniera asincrona, associando il codice ad un evento sul documento (e.g., il click su un bottone): event-oriented processing
- In maniera asincrona, associando il codice al completamento di un'operazione di rete.
- In maniera asincrona, associando il codice ad un timeout, i.e., un periodo di attesa dopo il quale lo script viene eseguito automaticamente

Durante l'esecuzione dello script, il browser è bloccato e non reagisce agli input dell'utente.

Come vediamo gli output?

- `document.write(string);` (scrivendo direttamente nella finestra del browser)
- `console.log(string);` (scrivendo sulla console)
- `alert(string);` (scrivendo in una finestra di alert)
- `document.getElementById(id).innerHTML = string;` (modificando il DOM del documento visualizzato)

7.1 Tipi di dato

JavaScript è minimale e flessibile per quel che riguarda i tipi di dati

- booleani
- numeri (interi e floating)
- stringhe
- null
- undefined
- *object* (unico tipo di dato strutturato, anche gli array ne fanno parte)
 - Gli object sono liste non ordinate di proprietà, coppie nome-valore.

```
let persona = {
  nome: 'Giuseppe',
  cognome: 'Rossi',
  altezza: 180,
  nascita: new Date(1995,3,12)
}
```

7.2 Oggetti Predefiniti

JavaScript predefinisce alcuni oggetti utili per raccogliere insieme i metodi più appropriati per certi tipi di dati.

- Oggetti multipli
 - Object
 - Array
 - String
 - Date
 - Number
 - RegExp
 - ⋮
- Oggetti singleton
 - Math
 - Json
 - ⋮

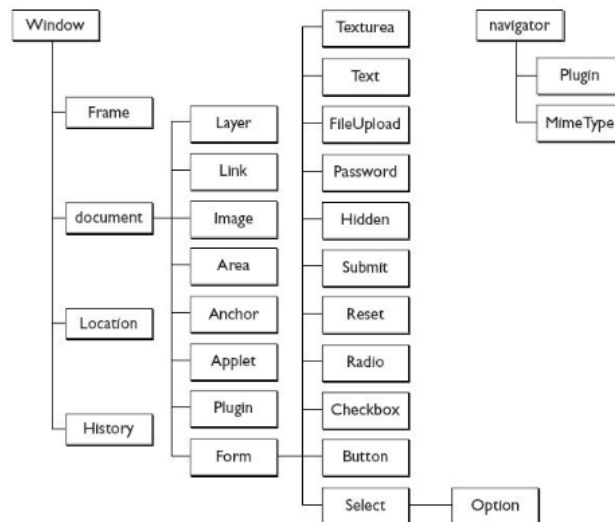
7.3 Json

JSON (JavaScript Object Notation) è un formato dati derivato dalla notazione usata da JS per gli oggetti.

```
{  
  "nome": ["Giuseppe", "Andrea", "Federico"],  
  "cognome": "Rossi",  
  "altezza": 180,  
  "nascita": "1995-04-11T22:00:00.000Z",  
  "indirizzo": {  
    "via": {  
      "strada": "Via□Indipendenza",  
      "numero": "15"  
    },  
    "citta": "Bologna",  
    "nazione": "Italia"  
  },  
  "telefono": [  
    { "tipo": "casa", "numero": "051□123456" },  
    { "tipo": "cell", "numero": "335□987654" }  
  ]  
}
```

Tutti i linguaggi di programmazione più importanti oggi accettano e scrivono dati in JSON.

7.4 Javascript client-side



- **window:**

- l'oggetto top-level con le proprietà e i metodi della finestra principale:

- * posizione: `moveBy(x,y)`, `moveTo(x,y)`, etc.
- * dimensioni: `resizeBy(x,y)`, `resizeTo(x,y)`, etc.
- * altre finestre: `open("URLname", "Windowname", ["opt"])`

- **navigator:**

- 'oggetto con le proprietà del client come nome, numero di versione, plug-in installati, supporto per i cookie, etc.

- **location**

- 'URL del documento corrente. Modificando questa proprietà il client accede a un nuovo URL (redirect):

- * `window.location = "http://www.cs.unibo.it/";`
- * `window.location.href = "http://www.cs.unibo.it/";`

- **history**

- 'array degli URL acceduti durante la navigazione. Possibile creare applicazioni client-side dinamiche che 'navigano la cronologia':

- * Proprietà: `length`, `current`, `next`
- * Metodi: `back()`, `forward()`, `go(int)`

- **document**

- appresenta il contenuto del documento, ed ha proprietà e metodi per accedere ad ogni elemento nella gerarchia:

- * `document.title`: titolo del documento
- * `document.forms[0]`: il primo form
- * `document.forms[0].checkbox[0]`: la prima checkbox del primo form

- * document.forms[0].check1: l'oggetto con nome "check1" nel primo form (non per forza una checkbox!)
 - * document.myform: l'oggetto "myform"
 - * document.images[0]: la prima immagine
- Inoltre document rappresenta l'oggetto DOMDocument del DOM del documento visualizzato

Modello di documento

Ogni oggetto nella gerarchia è caratterizzato da un insieme di proprietà, metodi ed eventi che permettono di accedervi, controllarlo, modificarlo.

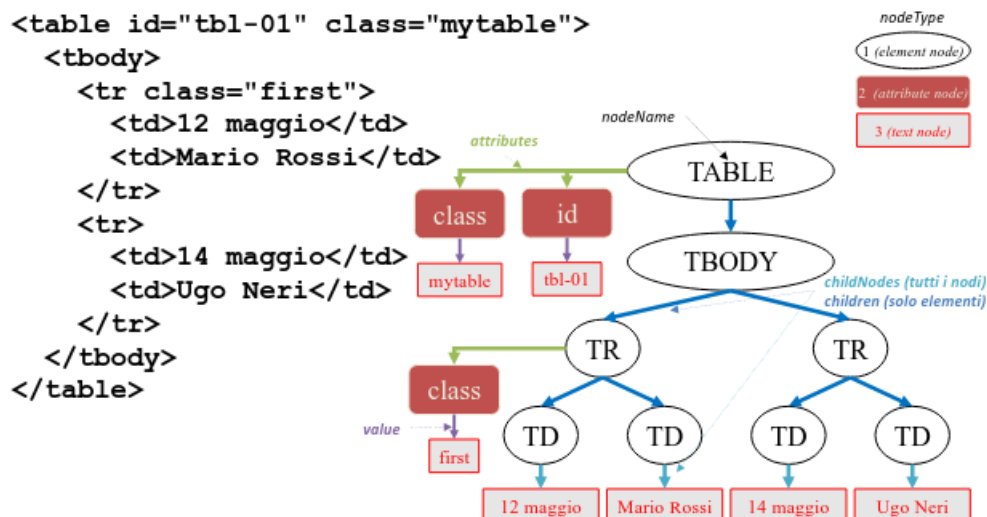
Javascript nacque per controllare i valori di un form:

```
function verify() {
    if (document.forms[0].elements[0].value == ""){
        alert("Il nome è obbligatorio!");
        document.forms[0].elements[0].focus();
        return false;
    }
    return true;
}
<form action= "... " onSubmit="Verify()">
<p>Name: <input type="text" name="userName"> </p>
```

7.5 Il Document Object Model (DOM)

Il Document Object Model è un'interfaccia di programmazione (API) per documenti sia HTML sia XML. Definisce la struttura logica dei documenti ed il modo in cui si accede e si manipola un documento.

Ogni componente di un documento HTML o XML può essere *letto*, *modificato*, *cancellato* o *aggiunto* utilizzando il Document Object Model.



7.6 Oggetti del DOM

Il core del DOM definisce alcune classi fondamentali per i documenti HTML e XML, e ne specifica proprietà e metodi. La classe principale di DOM è **DOMNode**, di cui la maggior parte

delle altre classi è una sottoclasse.

Le classi principali definiti nel DOM sono:

- **DOMDocument** : il documento di cui si sta parlando
- **HTMLElement**: ogni singolo elemento del documento
- **DOMAttr**: ogni singolo attributo del documento
- **DOMText**: ogni singolo nodo di testo del documento
- **DOMComment**, **DOMProcessingInstruction**, **DOMCDATASection**, **DOMDocumentType**, ecc.

7.7 DOM Node

DOMNode specifica i metodi per accedere a tutti gli elementi di un nodo di un documento, inclusi il nodo radice, il nodo documento, i nodi elemento, i nodi attributo, i nodi testo, ecc. Semplificando:

membri	metodi
– nodeName (<i>uppercase string</i>)	insertBefore()
– nodeType (<i>number</i>)	replaceChild()
– children (<i>array</i>)	removeChild()
– childNodes (<i>array</i>)	appendChild()
– parentNode (<i>elementNode</i>)	hasChildNodes()
– attributes (<i>array</i>)	hasAttributes()

7.8 DOM Element

HTMLElement specifica i metodi e i membri per accedere a qualunque elemento del documento. Semplificando

membri	metodi
nodeName	getAttribute()
	setAttribute()
	removeAttribute()

e analogamente per le altre classi ed interfaccia del DOM.

Javascript e DOM

Javascript implementa i metodi standard per accedere al DOM del documento.

```
var c = document.getElementById('c35');  
  
c.setAttribute('class','prova1');  
c.removeAttribute('align');  
var newP = document.createElement('p');
```

```

var text = document.createTextNode('Ciao Mamma.');
```

```

newP.appendChild(text);
c.appendChild(newP);
```

Creiamo una lista:

```

// Creazione elementi singoli
Olist = document.createElement("ol");
voce1 = document.createElement("li");
voce2 = document.createElement("li");
testo1 = document.createTextNode("un po' di testo");
testo2 = document.createTextNode("altro testo - item 2");

// Creazione lista completa
voce1.appendChild(testo1);
voce2.appendChild(testo2);
Olist.appendChild(voce1);
Olist.appendChild(voce2);

// Inserimenti lista in una data posizione
div = document.getElementById("lista");
body = document.getElementsByTagName("body").item(0);
body.insertBefore(Olist,div);
```

innerHTML e outerHTML

Il DOM per HTML (non generale!) permette di leggere/scrivere interi elementi, trattandoli come stringhe:

- **innerHTML**: legge/scrive il contenuto di un sottoalbero (escluso il tag dell'elemento radice)
- **outerHTML**: legge/scrive il contenuto di un elemento (incluso il tag dell'elemento radice)

7.9 Navigare sul DOM

- Il DOM (sia HTML sia XML) non ha metodi propri per navigare sull'albero liberamente.
- L'unico modo "ufficiale" è iterativamente esplorare l'albero attraverso la proprietà `children`.

```

//Trova la n-esima cella della m-esima riga della tabella con id dato
function getCell(tId,row,cell) {
    let items = document.body.children;
    let i = 0;
    do while (items[i].id !== tId) { i++ } ;
    let j = 0;
    do while (items[i].children[j].nodeName !== 'TBODY') {j++}
    let rows = items[i].children[j].children
    return rows[row].children[cell]
}
```

Selettori base HTML in DOM

I metodi base in DOM per accedere ai nodi di un documento sono:

- **getElementById**
 - solo ovviamente se l'elemento ha un id
- **getElementsByName**
 - se l'elemento ha un attributo name
- **getElementsByTagName**
 - tutti gli elementi con nome specificato

Javascript ed eventi DOM

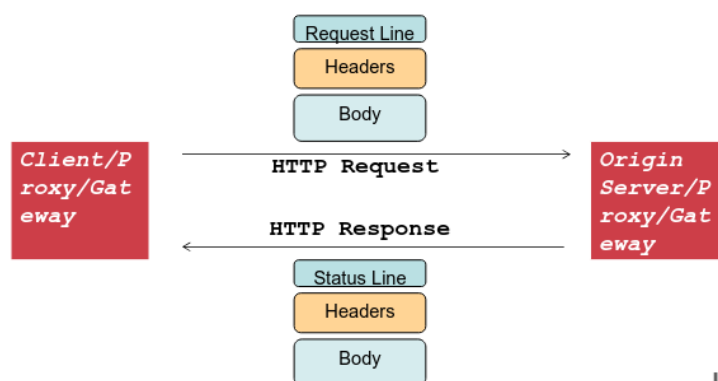
avascript permette di associare callback di eventi ad oggetti (dichiarazione locale o globale)

```
<script language="JavaScript">
  window.onkeypress= pressed;
  window.document.onclick = clicked;

  function pressed(e) { alert("Key␣pressed:␣" + e.which);}
  function clicked() { alert("Mouse␣Click!␣"); }
</script>
...
<body>
  <p id="mypara">Puoi
    <a href="test.htm" onClick="alert('Link!');">
      cliccare qui
    </a>
    oppure qui.
  </p>
</body>
```

Tutta la parte di sintassi/logica di JS non la riscrivo, la trovi in *18-Javascript-3.pdf*.

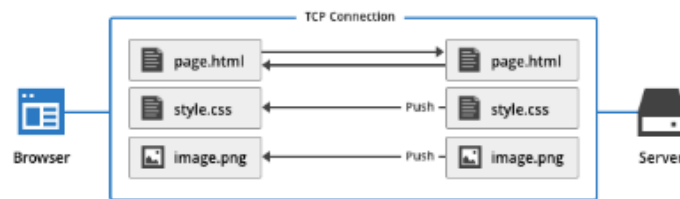
8 Approfondimenti su HTTP



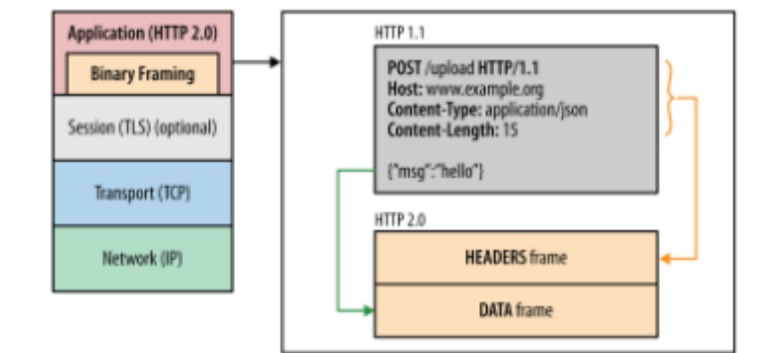
8.1 HTTP/2

Inizialmente chiamato HTTP 2.0 è la seconda major revision di HTTP. Non è una riscrittura del protocollo, i concetti principali e la compatibilità con HTTP 1.x restano: metodi, status code, headers, etc. Inizialmente proposto nel 2012 e standardizzato da IETF nel RFC 7540 a Maggio 2015 (HTTP1.1 è del 1997!).

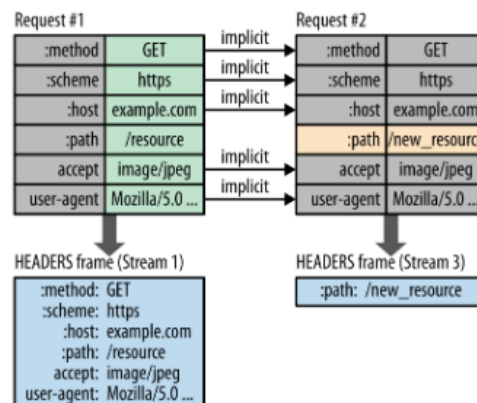
- HTTP/2 ha introdotto importanti novità mirate a migliorare le **performance**, tra cui:
 - **multiplexing**: nella stessa connessione è possibile avere richieste e risposte multiple, restituite anche in ordine diverso rispetto alle richieste e “ricostruite” nel client
 - supporto per operazioni di **Push**: il server può spedire più dati al client di quelli richiesti anticipando richieste successive nella stessa connessione



Concettualmente HTTP/2 è identico ad HTTP 1.1: ruoli, verbi, headers sono gli stessi. Tuttavia è un protocollo binario in cui i messaggi non sono più plaintext, ma codificati in maniera **compressa** e separando il blocco degli header dal payload. Ogni flusso di dati (stream) è suddiviso in frame separati e identificati che viaggiano anche sovrapposti (multiplexing).



L'algoritmo di compressione degli header HTTP/2 si chiama HPACK (RFC 7541). Solo gli header diversi da quelli delle richieste precedenti vengono inseriti nel frame e spediti. Secondo calcoli fatti, la compressione degli header ne riduce la dimensione del 30%.



8.2 HTTP/3

HTTP/3 è la versione più recente di HTTP, standardizzata nel 2023 come RFC 9114. L'obiettivo principale è aumentare ulteriormente le performance rispetto ad HTTP/2. Si basa sul protocollo QUIC proposto da Google nel 2012 e costruito su UDP (User Datagram Protocol), protocollo alternativo a TCP/IP. QUIC lavora a livello di trasporto e non di applicazione come HTTP.

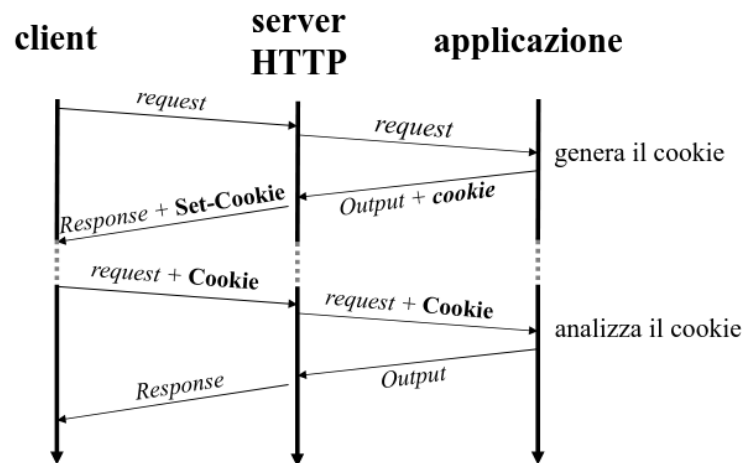
HTTP/3 quindi non si basa su TCP/IP, a differenza delle precedenti versioni, ma mantiene retrocompatibilità: metodi, header, status code restano invariati

8.3 Cookies

HTTP è **stateless**: non esiste nessuna struttura ulteriore alla connessione e il server non è tenuto a mantenere informazioni su connessioni precedenti.

- Un cookie (*concettualmente*) è una breve informazione scambiata tra il server ed il client

Il client mantiene alcune informazioni sulle connessioni precedenti, e le manda al server di pertinenza ogni volta che richiede una risorsa.



- Alla prima richiesta di uno user-agent, il server fornisce la risposta ed un header aggiuntivo, il cookie, con **dati arbitrari**, e con la specifica di usarlo per ogni successiva richiesta.
- Il server associa a questi dati ad informazioni sulla transazione.
- Ogni volta che lo user-agent accederà a questo sito, rifornirà i dati opachi del cookie che permettono al server di ri-identificare il richiedente

I cookies dunque usano due header, uno per la risposta, ed uno per le richieste successive:

- **Set-Cookie**: header della risposta, spedito dal server, il client può memorizzarlo e rispeditirlo alla prossima richiesta.
- **Cookie**: header della richiesta. Il client decide se spedirlo sulla base del nome del documento, dell'indirizzo IP del server, e dell'età del cookie.

8.4 Headers e struttura dei cookie

I cookies contengono dati arbitrari in formato testuale:

```
set-cookie: AWSALB=ojW4vtjxMz8WLY3jSrCUNekkG1aLZQHmb1SvExmvR6agqb+f+  
fC4RQWCb+  
gLViJpRakI8RrnfrXqiDmQ9KwqA8LiVMdhkBRUCCt0gwx5JBxKLmIBQ7gnbIFIZo+;  
Expires=Wed, 01 May 2000 11:07:46 GMT; Path=/
```

Oltre a nome e valore, includono informazioni che il client usa quando ri-spedisce il cookie:

- **Domain:** il dominio per cui il cookie è valido
- **Path:** l'URI per cui il cookie è valido
- **Max-Age/Expire:** La durata in secondi del cookie
- **Secure:** la richiesta che il client contatti il server usando soltanto un meccanismo sicuro (es. HTTPS) per spedirlo
- **Version:** La versione della specifica a cui il cookie aderisce.

Uso dei cookie

Esistono vari tipi di cookie usati per scopi diversi, tra cui:

- **Cookie persistenti:** hanno una validità temporale lunga, o addirittura non scadono mai, e sono usati per mantenere informazioni (semi)permanenti ad esempio informazioni su login, preferenze degli utenti, ecc.
- **Cookie di sessione:** hanno una durata breve e sono usati per raggruppare operazioni in sessioni di lavoro; contengono un identificativo della sessione le cui informazioni sono sul server; solitamente sono cancellati alla chiusura del browser
 - Autenticazione: processo di verifica dell'identità di un utente
 - * Session-based: il server memorizza un ID di sessione e informazioni sull'utente verificate ad ogni richiesta
 - * Token-based: il client memorizza un token, ricevuto dal server, che spedisce ad ogni richiesta ed è usato per la verifica
 - Autorizzazione: processo di verifica dell'effettiva possibilità di un utente di eseguire un'operazione e/o accedere a una risorsa
- **Cookie di terze parti:** appartengono ad un dominio diverso rispetto a quello della pagina in cui sono caricati; usati ad esempio per banner pubblicitari, permettono di ricostruire la navigazione degli utenti e possono essere usati in modo improprio; le impostazioni del browser permettono infatti di disabilitarli

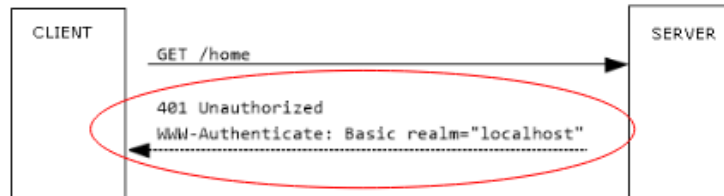
Header WWW-Authenticate

Quando l'utente prova ad eseguire un'azione che richiede autenticazione, il server risponde con uno status code **401** (Unauthorized) e aggiunge un header *WWW-Authenticate* in cui specifica i criteri da usare per l'autenticazione

Gli schemi più comuni:

- **Basic (disuso):** prevede la spedizione delle informazioni di autorizzazione in chiaro ad ogni risposta

- Digest: Il client spedisce nell'header Authorization una fingerprint della password, ovvero la password crittografata con il metodo MD5 (RFC 1321).
- Bearer (usato anche per autenticazione token-based): Nello schema Bearer (RFC 6750) il client non spedisce la password, in chiaro o cifrata, ma un token ("bearer token") che gli permette di accedere ad una risorsa



8.5 Cookie di sessione

I cookie di sessione contengono un ID univoco di sessione che il server ha memorizzato e riconosce. Il server si occupa anche di gestire la data di expiry (scadenza) del cookie e quindi chiusura della sessione.



Esempio in php:

```

<?php
session_start();

if( isset( $_SESSION['counter'] ) ) {
    $_SESSION['counter'] += 1;
}else {
    $_SESSION['counter'] = 1;
}

$msg = "You have visited this page ". $_SESSION['counter'];
$msg .= "in this session.";
?>

```

Esempio in Express.js:

```
// Use the session middleware
app.use(session({ secret: '<secret>', cookie: { maxAge: 60000 } }))

// Access the session as req.session
app.get('/', function(req, res, next) {
  if (req.session.views) {
    req.session.views++
    res.setHeader('Content-Type', 'text/html')
    res.write('<p>views: ' + req.session.views + '</p>')
    res.write('<p>expires in: ' + (req.session.cookie.maxAge / 1000) + 's</p>')
    res.end()
  } else {
    req.session.views = 1
    res.end('welcome to the session demo. refresh!')
  }
})
```

8.6 Base 64

Viene identificato un sottoinsieme di 64 caratteri di US-ASCII sicuri, che hanno la stessa codifica in tutte le versioni di ISO 646.

Ogni flusso di dati viene suddiviso in blocchi di 24 bit (3 byte). A loro volta questi 24 bit sono suddivisi in 4 blocchi di 6 bit ciascuno e codificati secondo una tabella prefissata in uno dei 64 caratteri.

La decodifica di Base64 è algoritmica, banale, non usa chiavi né calcoli di particolari complessità. NON È una tecnica crittografica!

Input	M						a						n											
Codice ASCII	77						97						110											
mappa bit	0	1	0	0	1	1	0	1	0	1	1	0	0	0	0	1	0	1	1	0	1	1	0	
Indice 6-bit	19						22						5						46					
Output Base64	T						W						F						u					
Codice ASCII	84						87						70						117					

8.7 Header e Payload

- **Header:** specifica il tipo di token e l'algoritmo di cifratura utilizzato
- **Payload:** informazioni di interscambio organizzate in affermazioni (claims) che possono essere di tre tipi:
 - Registered: claim predefiniti utili per descrivere il token
 - Public: arbitrari ma dichiarati nello IANA JSON Web Token Registry per evitare conflitti
 - Private: arbitrari e personalizzabili, utili per scambiare dati tra applicazioni che si accordano sui dati da usare

8.8 Signature

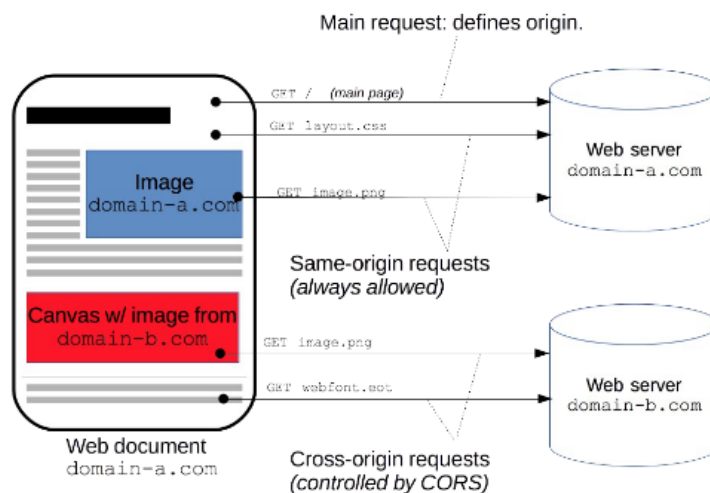
Il token (header e payload, in base64) può essere firmato con una chiave segreta lato server in modo da poterlo verificare nelle successive richieste e, se corrotto, non considerarlo valido. Cifratura simmetrica e asimmetrica (es. SHA256). **Il contenuto del token non è cifrato ma solo codificato in base64!** Si può decodificare facilmente, non bisogna quindi inserire dati sensibili.

8.9 CORS

Supponiamo che in una pagina appartenente ad un dominio protetto (es. in una sessione autenticata) si riesca ad intrufolare un pezzo di codice malizioso, che raccoglie e trasmette informazioni ad un repository accessibile ai malintenzionati.

- Si parla di vulnerabilità tra domini o **cross-domain**

Una delle possibili soluzioni, chiamata CORS, si basa sull'uso del metodo OPTION per indicare i domini ammessi.



Cross-Origin Resource Sharing: Una tecnica tutta HTTP introdotta dal W3C (W3C Rec 29/1/2013).

Si attiva solo per le connessioni Ajax (non si usa per connessioni HTTP normali) e prevede l'uso di due nuovi header:

- nella richiesta: **Origin**, per specificare il dominio su cui si trova il contenuto
- nella risposta: **Access-Control-Allow-Origin** per indicare gli altri domini da cui è permesso caricare contenuti

8.10 HTTP Caching

HTTP offre sofisticati meccanismi di caching, molto utili in applicazioni RESTFUL. Può essere client-side, server-side o su un proxy (intermedia).

La cache server-side riduce i tempi di computazione di una risposta, ma non ha effetti sul carico di rete, le altre riducono il carico di rete.

HTTP 1.1 introduce due tipi di meccanismi per controllare la validità dei dati in cache (cache control):

- Server-specified expiration
 - Il server indica una scadenza della risorsa. Può usare due meccanismi: header Expires o direttiva max-age in Cache-Control.
 - Se la data di scadenza è già passata, la richiesta deve essere rivalidata.
- Heuristic expiration
 - Poiché molte pagine non conterranno valori espliciti di scadenza, la cache stabilisce valori euristici di durata delle risorse, dopo le quali assume che sia scaduta. Usa informazioni contenute in altri header, ad esempio *Last-Modified time*
 - L'algoritmo esatto non è fissato nelle specifiche di HTTP ma dipende dall'implementazione